

FUNNY LAND

작성자 : 안승현

목차

- Funny Land 소개
- Photon API 소개
- Photon PUN 특징
- Photon PUN 구조
- 계획
 - 1주차
 - 2주차
 - 3주차
 - 4주차
- 개발에 사용된 에셋
- 구현에 어려웠던 점
- 해결
- 개발 후 느낀점 혹은 추후 개선 방향

FUNNY LAND 소개

- UNITY ENGINE (2020.3.19f) 사용
- URP(Universal Render PipeLine)라는 랜더링 방식 채택
- 멀티플레이 게임으로 매판 마다 호스트가 미니게임을 골라 플레이하는 게임입니다.
- 최종 라운드 까지 다른 플레이어 보다 많은 점수를 얻은 플레이어가 승리합니다.
- Photon API로 네트워크 기능 구현
- 유료 에셋 및 무료 에셋 사용
- 최대 10명까지의 멀티플레이 지원
- 실시간 네트워크 서비스
- <https://youtu.be/G7dX7UDZi8I> (YouTube 영상)



PHOTON API 소개

- 네트워크 게임 엔진
- Photon 에서 제공하는 API, 서버를 통해 코어한 부분을 잘 알지 못해도 네트워크 게임을 손쉽게 만들 수 있습니다.
- 서버인원은 무료는 10~20명 지원, 유료는 100이상 지원
- 서비스 종류는 pun, bolt, quantum 등 이중 PUN을 이용해 포트폴리오 제작



PHOTON PUN 특징

- PUN (Photon Unity Networking)의 약자
- 유니티에서 네트워크 동기화, RPC를 초보자가 쉽게 사용할 수 있도록 해준다.
- 클라이언트 서버 구조
- 방을 생성한 Master Client 서버, 호스트 둘 다 속하지 않는다.
- Master Client는 특별한 권한만 지니고 있다. (ex : 방장)
- Master Client가 접속을 종료하면 다른 Client한테 권한이 양도 된다.

PHOTON PUN 구조

Client
Client
Client
Client



Name Server

APP ID, Game Ver, 지역



Master Server

Looby, 방 생성 입장

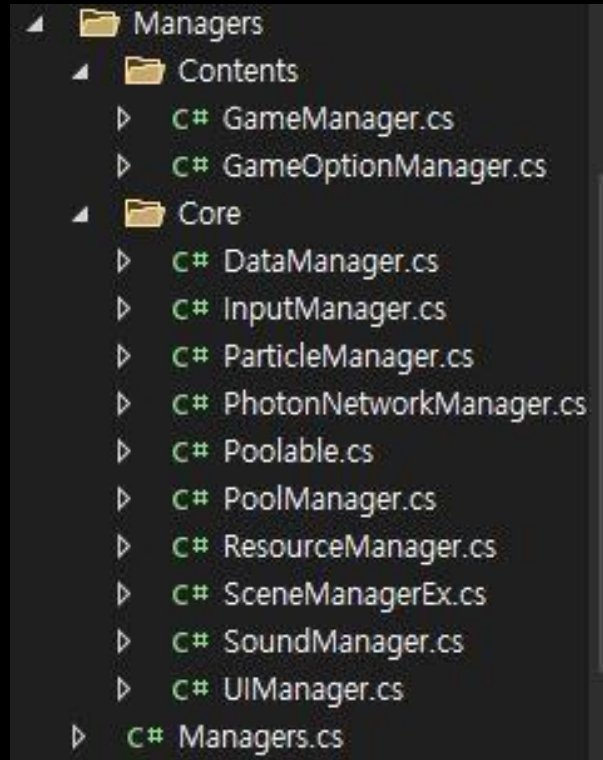


Game Server

멀티 플레이 콘텐츠

1주차 – 구조 설계

- Object Pooling(오브젝트 풀링), Data(데이터), Sound(소리), UI(사용자 인터페이스) 등등 자주 사용하는 클래스를 만들어 Manager 클래스의 멤버 변수로 만들고 나서 Manager 클래스를 싱글톤 만들어 접근하기 쉽게 설계
- Photon Api 구조 파악 후 씬을 로그인, 로비, 로딩, 게임, 콘텐츠 씬으로 나누고 게임씬에서 콘텐츠씬을 추가적으로 불러오는 방식으로 구현 왜냐하면 게임씬에서 공통적으로 사용하는 UI 및 기능을 재사용하기 위함
- BackEnd API를 사용하여 사용자 정보를 저장하는 DB 구현
- 플레이어 즉 클라이언트는 하나의 오브젝트만 소유하고 조종할 수 있다는 전제하에 코드 작성




```

public class Managers : MonoBehaviourPunCallbacks
{
    static Managers instacne;
    public static Managers Instance
    {
        get
        {
            Init();
            return instacne;
        }
    }

    #region Contents
    GameCursorManager gameCursor = new GameCursorManager();
    GameOptionManager setting = new GameOptionManager();
    GameManager game = new GameManager();
    QuestSystem quest = new QuestSystem();

    public static GameCursorManager GameCursor { get { return Instance.gameCursor; } }
    public static GameOptionManager Setting { get { return Instance.setting; } }

    public static GameManager Game { get { return Instance.game; } }

    public static QuestSystem Quest { get { return Instance.quest; } }
    #endregion

    #region Core
    DataManager data = new DataManager();
    InputManager input = new InputManager();
    PhotonNetworkManager photon = new PhotonNetworkManager();
    PoolManager pool = new PoolManager();
    ResourceManager resource = new ResourceManager();
    SceneManagerEx scene = new SceneManagerEx();
    SoundManager sound = new SoundManager();
    ParticleManager particle = new ParticleManager();
    UIManager ui = new UIManager();

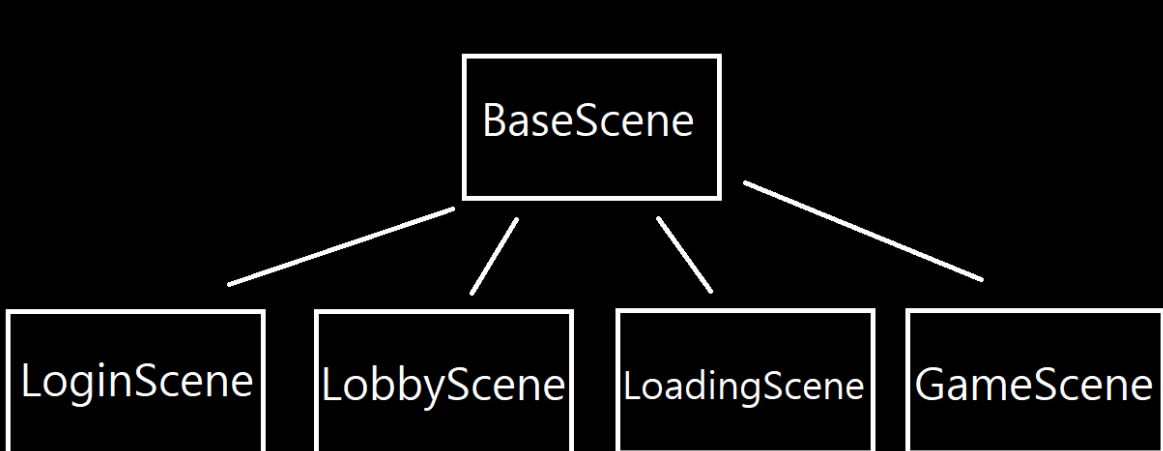
    public static DataManager Data { get { return Instance.data; } }
    public static InputManager Input { get { return Instance.input; } }
    public static PhotonNetworkManager Photon { get { return Instance.photon; } }
    public static PoolManager Pool { get { return Instance.pool; } }
    public static ResourceManager Resource { get { return Instance.resource; } }
    public static SceneManagerEx Scene { get { return Instance.scene; } }
    public static SoundManager Sound { get { return Instance.sound; } }
    public static ParticleManager Particle { get { return Instance.particle; } }
    public static UIManager UI { get { return Instance.ui; } }
    #endregion
}

```

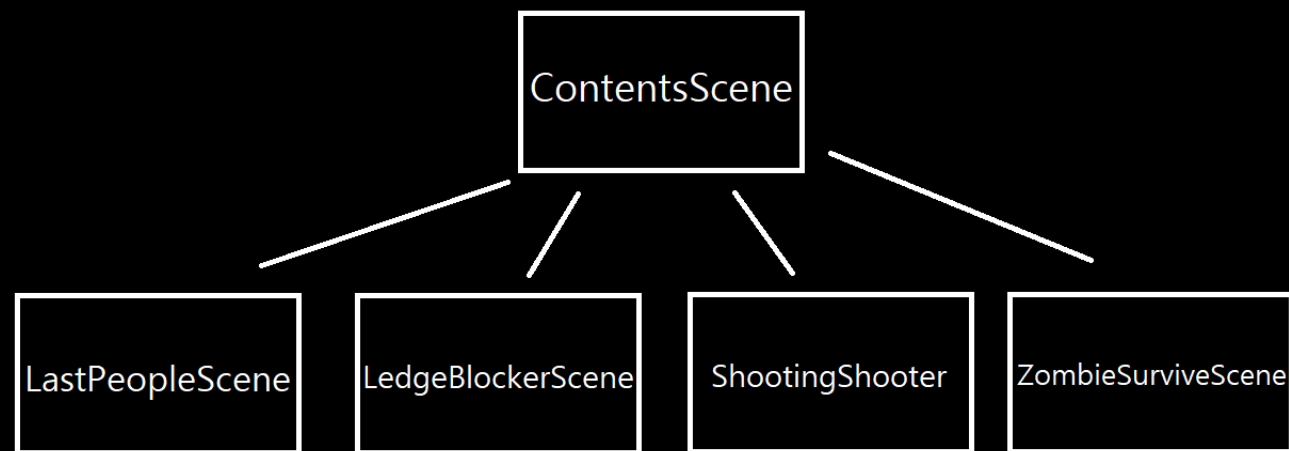
1주차 – 구조 설계

- 매니저 클래스
 - 싱글톤으로 구현하여 인스턴스 생성 비용 절감 및 접근성 향상
 - 매니저를 관리하는 매니저 클래스
 - 각 클래스의 기능을 나눠서 static 변수로 정의

1주차 - 구조 설계

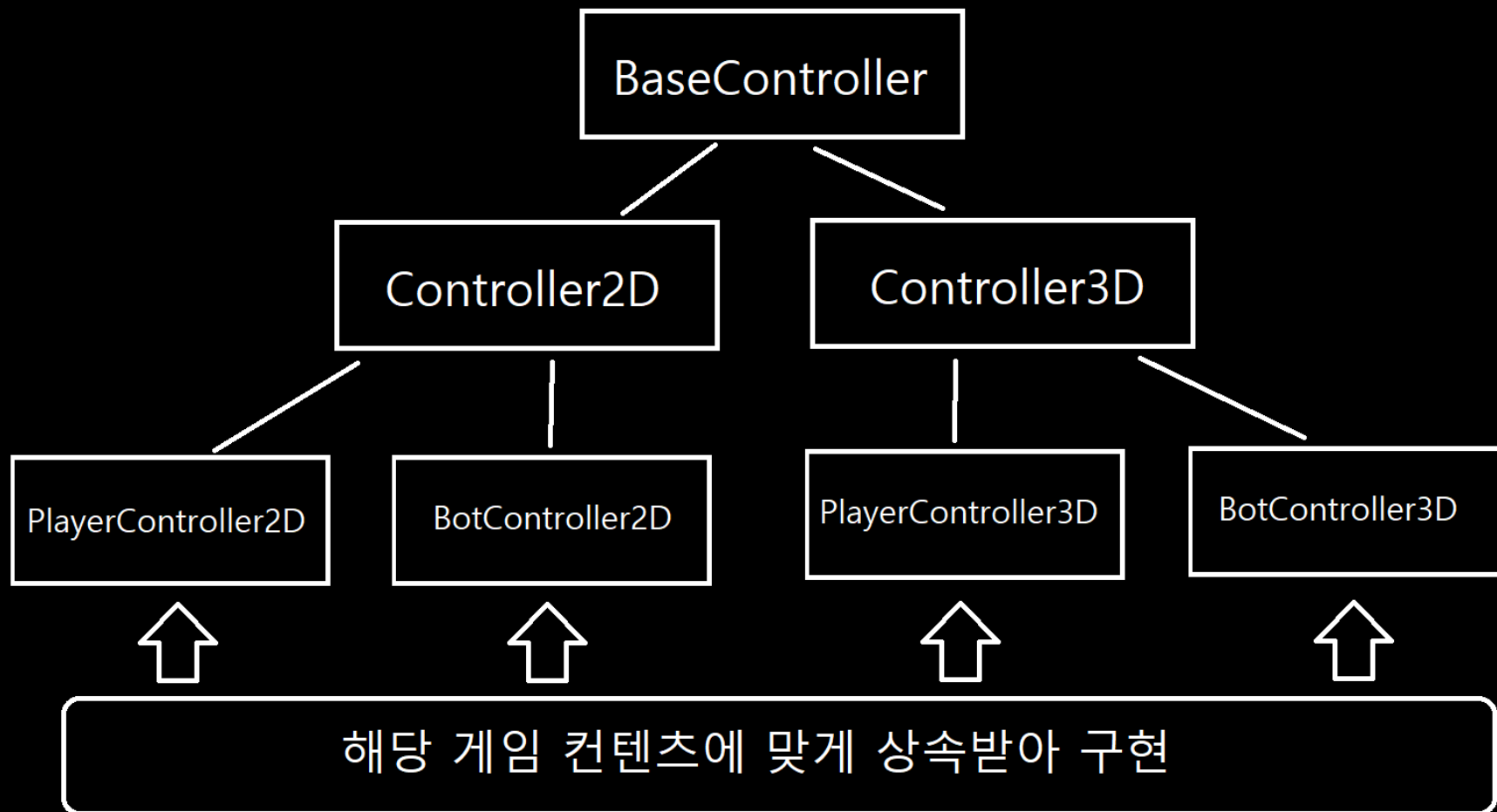


씬 클래스 구조

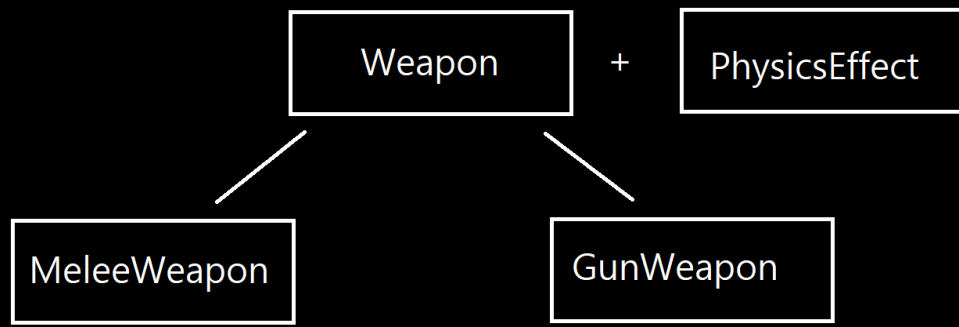


컨텐츠 씬 클래스 구조

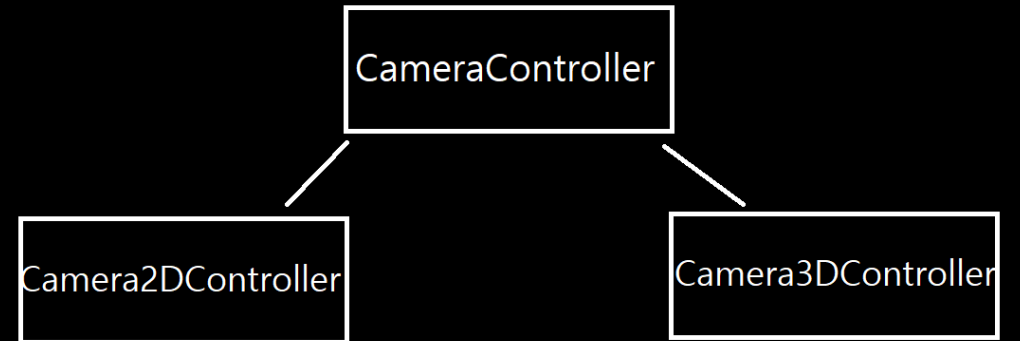
1주차 - 구조 설계



1주차 - 구조 설계



무기 클래스 구조



카메라 클래스 구조

1주차 – 구조 설계



로그인 씬

로그인 씬

- 게임 실행 시 처음 보여지는 씬
- 회원가입, 로그인, 아이디 찾기, 비밀번호 찾기, 환경설정을 할 수 있습니다.

1주차 – 구조 설계

```
void LoginBtnClick(PointerEventData data)
{
    Managers.Sound.Play2D("SFX/UI_Click");
    string id = Get<InputField>((int>Login.InputField_ID).text;
    string pw = Get<InputField>((int>Login.InputField_PW).text;
    bool isSaveID = Get<Toggle>((int>Toggles.IdRememberToggle).isOn;

    var result = Backend.BMember.CustomLogin(id, pw); ← 로그인 시도

    if (result.IsSuccess())
    {
        if (Managers.Data.PlayerInfoInit() && Managers.Data.PlayerInventoryInit())
        {
            if (isSaveID)
                PlayerPrefs.SetString("ID", id);
            else
                PlayerPrefs.DeleteKey("ID");

            Managers.Scene.LoadScene(Define.Scene.Lobby);
        }
        else
        {
            Managers.Scene.LeaveLobby();
        }
    }
    else
    {
        Managers.Sound.Play2D("SFX/UI_Error");
        Util.SimplePopup("ID 또는 PW가 존재하지 않거나, 잘못 입력되었습니다.");
    }
}
```

로그인 함수

- InputField에 있는 id, pw의 text를 가져와 Backend의 CustomLogin 함수에 id, pw를 매개변수를 넣어 로그인을 시도합니다.
- 로그인 성공시 플레이어 정보, 인벤토리를 초기화 하고 Lobby씬으로 이동합니다.

1주차 – 구조 설계

계정 생성

- InputField에 있는 email, id, pw의 text를 가져와 변수에 저장합니다.
- 사용 가능한 이메일인지 FindCustomID를 호출하여 결과를 bool형태로 반환합니다.
- 사용 가능한 이메일이면 CustomSignUp을 호출해 아까 입력해둔 id, pw를 매개변수로 넣어줍니다.
- 계정 생성이 완료되면 국가등록, 이메일등록, 성공알림 함수를 호출합니다.

```
void CreateAccount()
{
    string email = Get<InputField>((int)InputFields.InputField_Email).text;
    string id = Get<InputField>((int)InputFields.InputField_ID).text;
    string pw = Get<InputField>((int)InputFields.InputField_PW).text;

    if (isValidEmail(email))
        return;
    SetAccount(email, id, pw);
}
```

```
bool isValidEmail(string email)
{
    // 이메일 중복 확인
    var result = Backend.BMember.FindCustomID(email);
    if (result.IsSuccess())
    {
        Managers.Sound.Play2D("SFX/UI_Error");
        FailSign("중복하는 계정 (404)");
    }

    return result.IsSuccess();
}
```

```
void SetAccount(string email, string id, string pw)
{
    var result = Backend.BMember.CustomSignUp(id, pw);

    if (result.IsSuccess())
    {
        // 1. 국가등록
        SetCountry();

        // 2. 이메일 등록
        SetEmail(email);

        // 가입 성공 알림
        SuccessSign();
    }
    else
    {
        // 가입 실패 알림
        FailSign("회원 가입 실패 \n(중복된 ID 또는 네트워크 에러)");
    }
}
```

```
bool SetCountry()
{
    var result = Backend.BMember.UpdateCountryCode(Backend.GlobalSupport.CountryCode.SouthKorea);
    if (!result.IsSuccess())
    {
        FailSign("국가 설정 에러(204)");
    }

    return result.IsSuccess();
}

bool SetEmail(string email)
{
    var result2 = Backend.BMember.UpdateCustomEmail(email);
    return result2.IsSuccess();
}

void SuccessSign()
{
    Managers.Sound.Play2D("SFX/UI_Success");
    Util.SimplePopup("회원 가입 성공");
    gameObject.SetActive(false);
}

void FailSign(string message)
{
    Managers.Sound.Play2D("SFX/UI_Error");
    Util.SimplePopup(message);
}
```


1주차 – 구조 설계

```
public Dictionary<int, PlayerObject> _playersObjects { get; private set; } = new Dictionary<int, PlayerObject>();
public Dictionary<int, GameObject> _roomObjects { get; private set; } = new Dictionary<int, GameObject>();

#region 오브젝트 서치
public void SetPhotonObject(GameObject go)
{
    if(go.TryGetComponent(out PhotonViewEx PV))
    {
        if(PV.IsRoomView) // 룸 오브젝트
            SetRoomObj(go);
        else // 플레이어 오브젝트
            SetPlayer(go);
    }
}

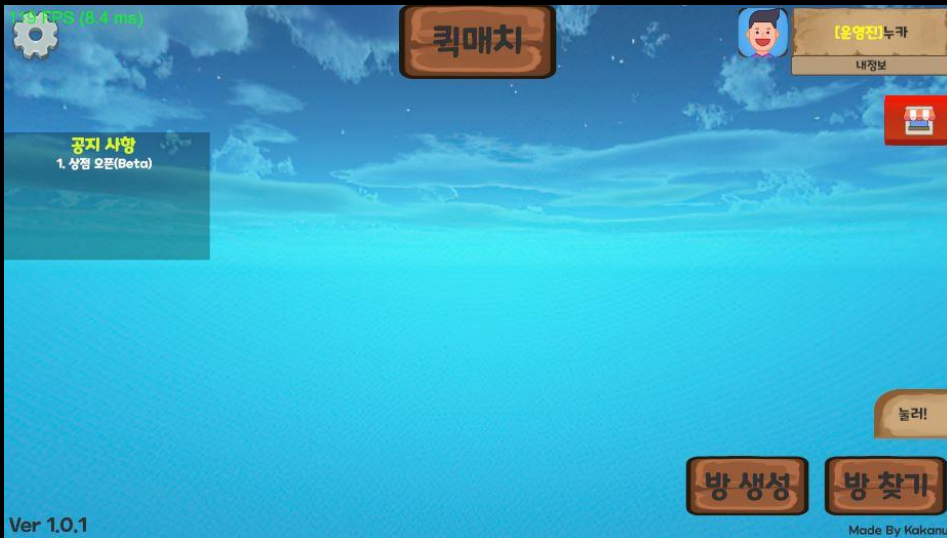
public GameObject GetPhotonObject(int viewID)
{
    if (viewID < 1000) // 룸 오브젝트
        return GetRoomObj(viewID);
    else // 플레이어 오브젝트
        return GetPlayerObject(viewID);
}

public string GetObjectNickName(int viewID)
{
    if (viewID < 1000) // 룸 오브젝트
        return GetRoomObj(viewID).name;
    else // 플레이어 오브젝트
        return GetPlayer(viewID).NickName;
}
}
#endregion
```

GameManager

- 네트워크에 존재하는 게임 오브젝트를 쉽게 서치하기 위한 클래스
- 네트워크 오브젝트들은 고유의 ViewID를 가지고 있어 이를 key로 이용해 자료구조 Dictionary에 값을 저장
- BaseController를 상속받는 모든 클래스가 Awake() 실행 시 SetPhotonObject함수 실행하여 등록하도록 설계
- Photon은 viewID 0 ~ 999까지 룸 오브젝트 1000번 이후로 플레이어당 1000 개의 오브젝트들을 각각 할당 할 수 있습니다.

1주차 – 구조 설계

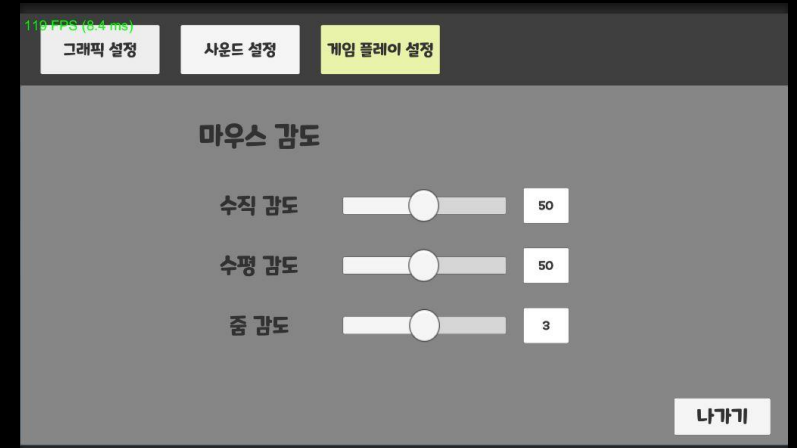
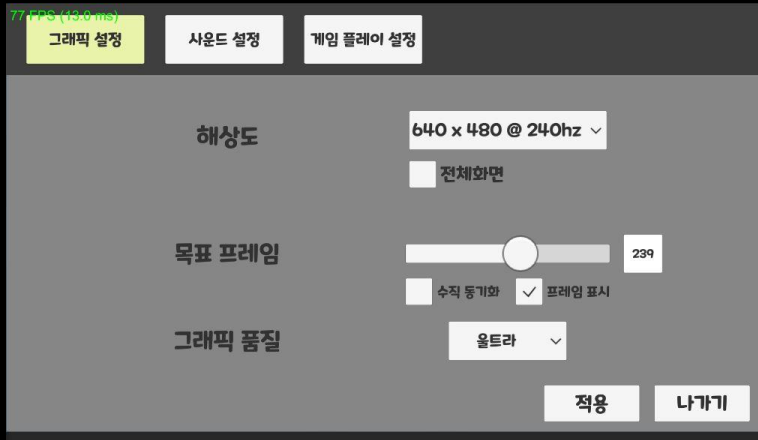


로비 씬

로비 씬

- 로그인을 성공하면 진입하는 씬
- 방 생성, 방 찾기, 퀵매치, 내정보, 상점, 환경설정을 할 수 있는 씬 입니다.

1주차 – 구조 설계

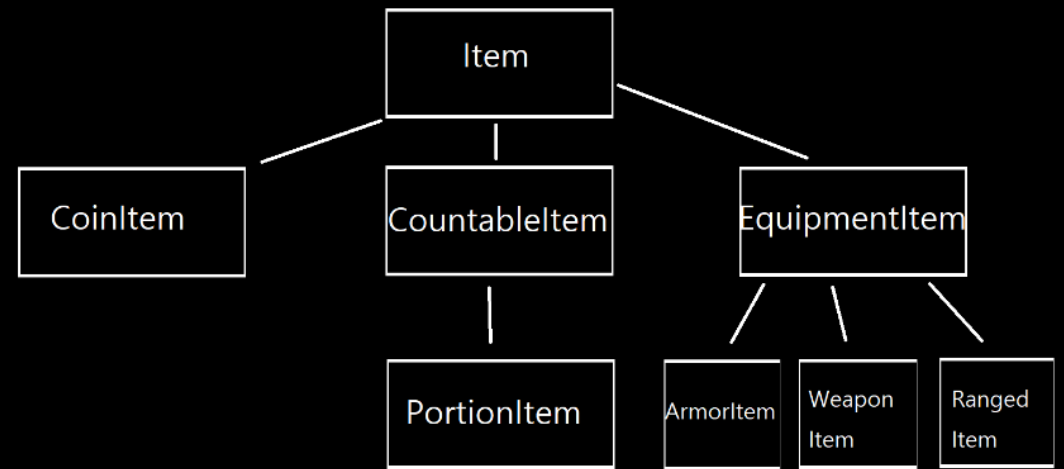


환경 설정

- 게임 해상도, 그래픽 품질, 사운드를 설정할 수 있는 창 입니다.

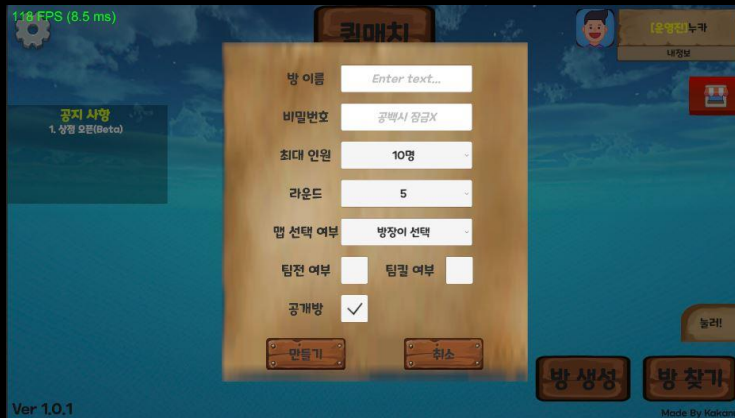
2주차 – 방, 방목록, 내 정보 및 인벤토리 구현

- 플레이어 정보를 담는 PlayerInfo 클래스 구현
- 아이템 구현
- 사용자 지정 방 구현
- 사용자 지정 방목록 구현

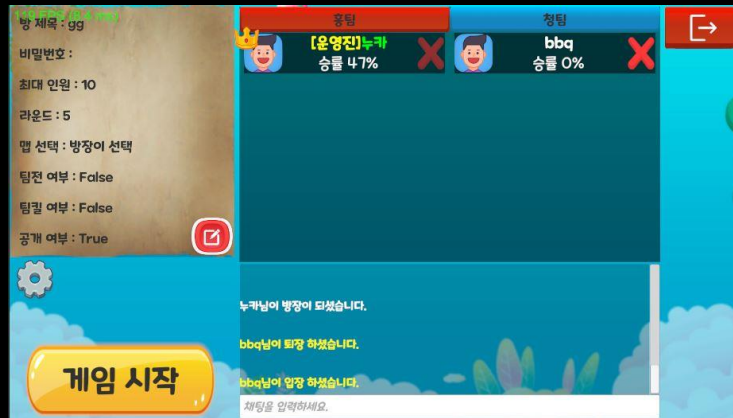


아이템 클래스 구조

2주차 – 방, 방목록,내 정보 및 상점 구현



방 생성



생성된 방



방 목록

2주차 – 방, 방목록, 내 정보 및 상점 구현

```
void CreateRoom()
{
    string roomName = Get<InputField>((int)InputFields.RoomNameInputField).text;
    if(roomName == "")
    {
        Managers.Sound.Play2D("SFX/UI_Error");
        Util.SimplePopup("방 이름은 공백으로 두 수 없습니다.");
        return;
    }

    string _roomName = Get<InputField>((int)InputFields.RoomNameInputField).text;
    string _pw = Get<InputField>((int)InputFields.RoomLockeInputField).text;
    int _curRound = 1;
    int _round = int.Parse(Get<Dropdown>((int)Dropdowns.GameRoundDropdown).captionText.text);
    string _mapSelect = Get<Dropdown>((int)Dropdowns.RoomMapSelectDropdown).captionText.text;
    bool _team = Get<Toggle>((int)Toggles.RoomTeamToggle).isOn;
    bool _teamKill = Get<Toggle>((int)Toggles.RoomTeamKillToggle).isOn;
    byte _maxPlayers = (byte)(Get<Dropdown>((int)Dropdowns.RoomPeopleDropdown).value + 2);
    bool _roomVisible = Get<Toggle>((int)Toggles.RoomVisibleSelectToggle).isOn;

    PhotonNetwork.CreateRoom(PhotonNetwork.LocalPlayer.UserId,
        Managers.Photon.InitRoomProperties(_roomName, _pw, _curRound, _round, _mapSelect, _team, _teamKill, _maxPlayers, _roomVisible));
}
```

방 생성

- 방 생성 전 옵션을 설정하는 단계 입니다. (방 이름, 비밀번호, 최대 라운드, 맵 선택 방식, 팀전 여부, 팀킬 여부, 최대 플레이어, 공개 여부)
- InputField에 있는 각각의 text를 가져와 각 변수에 값을 넣어 CreateRoom을 호출할 시 매개변수로 넣어줍니다.
- InitRoomProperties 함수는 사용자 지정 함수이며 RoomOption 객체를 생성해 Hashtable로 방 설정 과 로비에 보일 string key값을 설정하여 RoomOption을 반환

2주차 - 방, 방목록, 내 정보 및 상점 구현

```
private void Start()
{
    PV.RPC("RequestUIPos", RpcTarget.MasterClient, PhotonNetwork.LocalPlayer);
    StartCoroutine(CorRoomInfoInit());
}
```

```
[PunRPC]
void RequestUIPos(Player requestPlayer)
{
    int redCount = Get<Transform>((int)Transforms.RedTeam).childCount;
    int blueCount = Get<Transform>((int)Transforms.BlueTeam).childCount;

    if (redCount <= blueCount && Get<Transform>((int)Transforms.RedTeam).childCount < 5)
    {
        PV.RPC("SetUIPos", requestPlayer, true, "RedTeam");
    }
    else
    {
        PV.RPC("SetUIPos", requestPlayer, true, "BlueTeam");
    }
}
```

```
[PunRPC]
void SetUIPos(bool isInit, string parent)
{
    Managers.Photon.SetPlayerPropertie("Team", parent);

    if (isInit)
    {
        GameObject go = Managers.Resource.PhotonInstantiate(UI, Vector3.zero, Quaternion.identity, Vector3.one, Define.PhotonObjectType.PlayerObject, parent);
        myPlayer = go.GetComponent<UI_Player>();
    }
    else
    {
        myPlayer.GetComponent<PhotonViewEx>().RPC("RpcParent", RpcTarget.AllBuffered, parent);
    }
}
```

생성된 방

- Start() 함수 실행 시 유저는 마스터 클라이언트에게 플레이어 정보를 나타내는 UI를 생성이 되는지 요청을 합니다.
- RequestUIPos로 요청을 받을 후 현재 RedTeam, BlueTeam의 유저 UI 개수를 계산해 개수가 적은 쪽을 찾은 다음 해당 유저에게 해당 팀의 UI를 생성하도록 RPC를 합니다.
- SetUIPos에서 처음 생성시 PhotonInstantiate를 생성 후 myPlayer로 캐싱합니다. 아닐 경우 RpcParent를 호출하여 위치를 조정합니다.

2주차 – 방, 방목록, 내 정보 및 상점 구현

```
void OnPointerDown()
{
    if (!PhotonNetwork.IsMasterClient)
        return;
    Managers.Sound.Play2D("SFX/UI_Click");
    _beginParent = transform.parent;
    _beginIndex = transform.GetSiblingIndex();
    _beginPosition = transform.position;
    _beginMousePosition = Input.mousePosition;

    transform.SetParent(transform.parent.parent);
    transform.SetAsLastSibling();
}

void OnPointerDrag()
{
    if (!PhotonNetwork.IsMasterClient)
        return;

    transform.position = _beginPosition +
        (Input.mousePosition - _beginMousePosition);
}

void OnPointerUp()
{
    if (!PhotonNetwork.IsMasterClient)
        return;
    // 밖으로 뱉낸 상태가 아니면
    if (transform.parent != _beginParent.parent)
        return;

    Managers.Sound.Play2D("SFX/UI_Dropdown");
    transform.SetParent(_beginParent);
    transform.SetSiblingIndex(_beginIndex);
    UI_Room room = FindObjectOfType<UI_Room>();
    room.PointerUpPos(this);
}
```

```
public void PointerUpPos(UI_Player uI_Player)
{
    if (uI_Player == null)
        return;

    _ped.position = Input.mousePosition;
    _rcList.Clear();

    _gr.Raycast(_ped, _rcList);

    if (_rcList.Count == 0)
        return;

    foreach (var raycast in _rcList)
    {
        var team = raycast.gameObject.GetComponent<UI_TeamRange>();
        if (team)
        {
            MoveTeam(uI_Player.PV.Owner, team.gameObject.name);
            break;
        }
    }
}
```

```
[PunRPC]
void MoveTeam(Player requestPlayer, string team)
{
    int Count = 5;
    if (team == "RedTeam")
        Count = Get<Transform>((int)Transforms.RedTeam).childCount;
    else if (team == "BlueTeam")
        Count = Get<Transform>((int)Transforms.BlueTeam).childCount;

    if (Count < 5)
        PV.RPC("SetUIPos", requestPlayer, false, team);
}
```

방 플레이어 UI 이동

- 마스터 클라이언트만 이동 가능
- 마우스 클릭 시 OnPointerDown()호출 대상 UI를 클릭하면 UI가 다른 UI에 가리지 않도록 SetAsLastSibling() 호출
- 마우스 클릭 중 OnPointerDrag() 호출 해당 UI가 마우스 포인터를 따라 움직인다.
- 마우스 클릭을 뗔을때 OnPointerUp() 호출 PointerUpPos()를 통해 Raycast를 한 후 리스트 중 UI_TeamRange를 가진 컴포넌트랑 충돌 했을 시 해당 UI를 MoveTeam() 함수를 통해 이동 시킨다.

2주차 – 방, 방목록, 내 정보 및 상점 구현

```
#region 방리스트 관리
// 왼쪽(-2) 오른쪽(-1) 버튼자(그외)
void BtnClick(int num)
{
    if (num == -2) --curPage;
    else if (num == -1) ++curPage;
    else print(myRoomList[multiple + num]);

    RoomInit(myRoomList);
}

public void RoomInit(List<RoomInfo> roomList)
{
    myRoomList = roomList;

    maxPage = (roomList.Count % RoomCount == 0 ? roomList.Count / RoomCount : roomList.Count / RoomCount + 1);

    multiple = (curPage - 1) * RoomCount;

    Get<Button>((int)Buttons.LeftButton).interactable = (curPage <= 1) ? false : true;
    Get<Button>((int)Buttons.RightButton).interactable = (curPage >= maxPage) ? false : true;
    Get<Text>((int)Texts.PageText).text = $"{curPage}/{maxPage}";

    for (int i = 0; i < RoomCount; i++)
    {
        string roomName = "";
        string pw = "";
        string playerCount = "";
        bool isActive = false;

        if (i + multiple < roomList.Count)
        {
            Hashtable hs = roomList[i + multiple].CustomProperties;
            roomName = hs["RoomName"].ToString();
            pw = hs["Pw"].ToString();
            playerCount = $"{roomList[i + multiple].PlayerCount}/{roomList[i + multiple].MaxPlayers}";
            isActive = true;
        }

        Get<UI_RoomInfo>(i).Get<Text>((int)UI_RoomInfo.Texts.RoomNameText).text = roomName;
        Get<UI_RoomInfo>(i).Get<Image>((int)UI_RoomInfo.Images.LockImage).enabled = (pw != "" ? true : false);
        Get<UI_RoomInfo>(i).Get<Text>((int)UI_RoomInfo.Texts.CurPeopleText).text = playerCount;
        Get<UI_RoomInfo>(i).Get<Button>((int)UI_RoomInfo.Buttons.Button).interactable = isActive;
    }

    SetRoomInfo();
}
#endregion
```

방목록 리스트 초기화

- LobbyScene에서 포톤에서 제공되는 함수 OnRoomListUpdate(List<RoomInfo> roomList)를 통해 생성된 방 정보를 받아와 RoomInit을 호출합니다.
- curPage 현재 위치한 페이지를 나타내는 변수이다.
- maxPage 변수는 최대 방 목록 페이지 수를 나타내는 변수이며 roomList.Count % RoomCount의 나머지에 따라 값이 정해집니다.
- roomList는 방목록을 List로 담아둔 변수이다.
- RoomCount 한 페이지에 보여질 방 목록 개수를 나타내는 변수
- 반복문은 현재 페이지 보여질 방 개수에 맞게 순회하면서 방 정보를 업데이트를 합니다.

2주차 – 방, 방목록, 내 정보 및 상점 구현

```
#region 방 입장
void JoinRoom(int i)
{
    if (myRoomList[i + multiple] == null)
        return;
    Hashtable hs = myRoomList[i + multiple].CustomProperties;
    string pw = hs["PW"].ToString();
    bool isJoin = myRoomList[i + multiple].PlayerCount < myRoomList[i + multiple].MaxPlayers;

    if(isJoin)
    {
        if (!string.IsNullOrEmpty(pw))
        {
            UI_LobbyScene uIs = Managers.UI.SceneUI as UI_LobbyScene;
            uIs.roomPW = Managers.UI.ShowPopupUI<UI_RoomPW>("RoomPW");
            uIs.roomPW.RoomPWInit(myRoomList[i + multiple]);
        }
        else
        {
            PhotonNetwork.JoinRoom(myRoomList[i + multiple].Name);
        }
    }
    else
    {
        Managers.Sound.Play2D("SFX/UI_Error");
        Util.SimplePopup("방이 꽉 찼습니다.");
    }
}

public override void OnJoinedRoom()
{
    CloseAllPopupUI();
}

public override void OnJoinRoomFailed(short returnCode, string message)
{
    Managers.Sound.Play2D("SFX/UI_Error");
    Util.SimplePopup("방이 존재하지 않거나 \n 인원이 꽉찬 방 입니다.");
}
#endregion
```

방 입장

- 입장 하려고 하는 방의 인덱스로 리스트 배열의 방정보를 가져와 해당 정보가 존재하고 만약 패스워드가 존재 시 패스 워드 까지 입력 받고 방 최대 인원에 초과하는지 확인하고 조건 충족시 PhotonNetwork.JoinRoom()을 호출하여 방 입장

2주차 – 방, 방목록,내 정보 및 상점 구현



내 정보



상점

2주차 – 방, 방목록, 내 정보 및 상점 구현

```
public void ShowPurchaseWindow(int id, Text priceText)
{
    if (Managers.Data.ItemDict.TryGetValue(id, out Data.ItemData _ItemData))
    {
        UI_Notice notice = Util.SimplePopup($"[_ItemData.Name]를 구매하시겠습니까.\n 구매비용은 {_ItemData.Price} {Util.GetGoodsTypeToString(_ItemData.Goods)} 입니다.");
        notice.Get<Button>((int)UI_Notice.Buttons.OKButton).gameObject.CoverBindEvent((data) =>
        {
            Managers.Sound.Play2D("SFX/UI_Click", Define.Sound2D.Effect2D);

            if(Util.PurchaseItemResult(_ItemData.Goods, _ItemData.Price))
            {
                gameObject.RemoveAllEvent();
                notice.ClosePopupUI();
                Managers.Data.PlayerInventoryData.itemId.Add(_ItemData.Id);
                Managers.Data.SaveBackEndData(Managers.Data.PlayerInventoryData, "PlayerInventory", "playerInventoryToJson");
                Managers.Sound.Play2D("SFX/UI_PurchaseCompleted");
                Util.SimplePopup("구매이 완료되었습니다.");
                priceText.text = $"보유중";
            }
            else
            {
                Managers.Sound.Play2D("SFX/UI_AlertSound");
                Util.SimplePopup("비용이 부족합니다.");
            }
        });
        notice.ActiveCancelBtn();
    }
}
```

상점

- 아이템 구입시 해당 ID, 가격표로 창을 띄운 후 해당 금액이 맞으면 아이템을 추가하고 BackEnd에 데이터를 갱신

2주차 - 방, 방목록, 내 정보 및 상점 구현

```
Get<Text>((int)Texts.RaceResultText).text = $"전적#{Managers.Data.PlayerInfoData.Total}전 " +  
    $"#{Managers.Data.PlayerInfoData.win}승 " +  
    $"#{Managers.Data.PlayerInfoData.lose}패#{n} " +  
    $"승률 {Managers.Data.PlayerInfoData.WinRate}%";  
  
Get<InputField>((int)InputFields.IntroduceInputField).text = Managers.Data.PlayerInfoData.introduceText;  
Get<Text>((int)Texts.CoinText).text = Managers.Data.PlayerInfoData.coin.ToString();  
Get<Text>((int)Texts.DiaText).text = Managers.Data.PlayerInfoData.dia.ToString();
```

```
Managers.UI.SceneUI.UpdateSyncUI += SyncImage;  
Managers.UI.SceneUI.UpdateSyncUI += SyncNick;  
Managers.UI.SceneUI.UpdateSyncUI += SyncAsset;  
Managers.UI.SceneUI.UpdateSyncUI?.Invoke();
```

내 정보 표시

- UI 출력시 Start함수에서 초기화 수행
- BackEnd에서 불러온 데이터를 Text에 표시
- 델리케이트 UpdateSyncUI 를 사용해 데이터 수정 시 구독한 함수들을 실행

3주차 – 로딩썸, 게임썸, 콘텐츠 구현

- 방에서 방장이 게임 시작을 누르면 로딩썸으로 이동합니다.
- 모든 플레이어가 로딩썸에서 로딩을 마치면 게임썸으로 전환됩니다.
- 게임썸에서 방장이 60초 안에 원하는 미니 게임을 선택해 플레이 하면 됩니다.
- 최대 라운드는 게임 시작전 설정한 라운드를 따라가며 모든 라운드가 끝나면 개인전, 팀전에 따라 점수가 높은 유저 또는 팀이 승리합니다.

119 FPS (8.4 ms)
개인전

로딩 완료된 플레이어 0/2



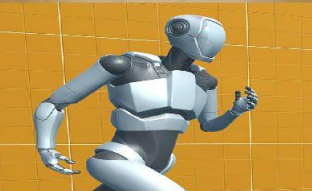
Tip : 요령껏 하세요.

95%
Now Loading..



119 FPS (8.4 ms)

방장이 선택 최후의 생존자 52
1/5

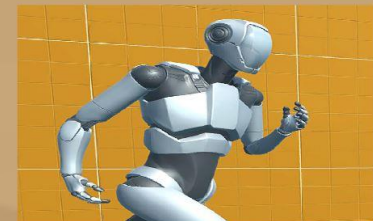


선택

118 FPS (8.4 ms)

최후의 생존자

생존



NPC를 흉내 내면서 유저를 찾아 잡아라



3주차 – 로딩썸, 게임썸, 콘텐츠 구현



게임썸

- 자주 쓰이는 기능을 게임 썸에 넣어 재사용성을 높였습니다.
- Tab키를 누르면 현재 플레이어의 점수와 순위가 표시가 됩니다
- ESC를 누르면 환경설정 및 게임종료를 할 수 있습니다.



3주차 – 로딩씬, 게임씬, 콘텐츠 구현



LastPeople(컨텐츠)

- 플레이어와 AI 사이에서 플레이어를 찾아 제거하면 되는 심플한 게임입니다.
- 매 시간마다 자기장이 줄어 듭니다. (최대 5 번)
- 자기장에 닿으면 화면이 파랗게 변하고 체력이 서서히 닳습니다.
- 플레이어가 아닌 AI를 제거할 시 자신의 체력이 닳습니다.
- 상대 플레이어를 모두 제거하면 게임이 종료 됩니다.
- 매 60초마다 랜덤으로 레드존이 발생합니다.

3주차 – 로딩씬, 게임씬, 컨텐츠 구현

```
protected override void UpdateState()
{
    Input();
    switch (State)
    {
        case Define.State.Idle:
            UpdateIdle();
            break;
        case Define.State.Moving:
            UpdateMoving();
            break;
        case Define.State.Run:
            UpdateRunning();
            break;
        case Define.State.NormalAttack:
            UpdateAttack();
            break;
        case Define.State.Jumping:
            UpdateJumping();
            break;
    }
}
```

```
protected override void UpdateState()
{
    int index = Random.Range(0, 100);

    if(index <= 50) // 아이들 50가지 경우의 수
        UpdateIdle();
    else if(index <= 90) // 워크 40가지 경우의 수
        UpdateMoving();
    else // 런 10가지 경우의 수
        UpdateRunning();
}
```

LastPeople FSM(컨텐츠)

- 플레이어와 AI 조작 로직은 FSM(유한 상태 머신)으로 구성되어 있습니다.
- 각 상태에 따라 유기적으로 연결되어 있습니다.

3주차 – 로딩씬, 게임씬, 콘텐츠 구현

```
protected override void OnAttack()
{
    if (!PhotonNetwork.IsMasterClient || bc.stat.Hp <= 0)
        return;

    curCount = 0;
    RaycastHit[] hits = Physics.BoxCastAll(transform.position + localAtkPos, attackSize / 2f, transform.forward, transform.rotation, attackRange);
    foreach (RaycastHit hit in hits)
    {
        if (hit.collider.gameObject == this.gameObject)
            continue;

        if (curCount == multiAtkCnt)
            break;

        if (hit.transform.TryGetComponent(out BaseController target))
        {
            if (target.stat.Hp <= 0)
                continue;

            if (!target.PV.IsRoomView)
            {
                if (Managers.Game.IsTeamMode && !Managers.Game.IsTeamKill)
                {
                    if (target.team == bc.team)
                        continue;
                }
            }
            else
            {
                if (target.team == bc.team)
                    continue;
            }

            OnAdditionPhysiced(target.PV);
            target.PV.RPC("OnAttacked", target.PV.Owner, bc.PV.ViewID);
            curCount++;
        }
    }
}
```

LastPeople 플레이어 공격(컨텐츠)

- 플레이어 공격은 Dead상태가 아니라면 Attack상태로 전환이 가능합니다.
- 공격시 마스터 클라이언트만 피격 판정을 합니다.
- BoxCastAll을 통해 피격된 모든 게임 오브젝트를 순회합니다.
- 현재 공격 카운트가 최대 카운트랑 같으면 더 이상 반복문을 순회하지 않습니다.
- 팀 여부 확인, 추가 효과 확인후 해당 소유자에게 OnAttacked함수(피격 함수) 호출 합니다.

3주차 – 로딩씬, 게임씬, 컨텐츠 구현

```
public bool PageStart()
{
    if (pageSize.Count <= CurPage + 1)
        return false;

    if (pageSize.Count - 1 == CurPage + 1)
        gameObject.GetComponent<PhotonViewEx>().Rpc2DSound("BGM/LastPage", (int)Define.Sound2D.Bgm, 1f);

    StopCoroutine(PageCoroutine());
    NextDestination();
    StartCoroutine(PageCoroutine());
    Managers.Game.Scene.PV.RPC("RegisterMessage", RpcTarget.AllViaServer, $"<color=red>주의!</color> \n자기장이 켜집니다.({curPage} 페이지)", 5f);
    return true;
}

void NextDestination()
{
    int nextPage = CurPage + 1;

    if (pageSize[nextPage] == 0f)
        nextPagePos = transform.position;
    else
    {
        float curRadius = pageSize[CurPage] * 0.5f; // 현재 페이지 반지름 추출
        float nextRadius = pageSize[nextPage] * 0.5f; // 다음 페이지 반지름 추출

        float gap = curRadius - nextRadius;

        Vector3 nextPos = new Vector3(
            transform.position.x + Random.Range(-gap, gap),
            0,
            transform.position.z + Random.Range(-gap, gap)
        );

        nextPagePos = nextPos;
    }

    nextPageSize = new Vector3(pageSize[nextPage], transform.localScale.y, pageSize[nextPage]);
    CurPage++;
}

IEnumerator PageCoroutine()
{
    isStart = true;
    while ((transform.position - nextPagePos).magnitude > 0.1f || (transform.localScale - nextPageSize).magnitude > 0.1f)
    {
        transform.position = Vector3.Lerp(transform.position, nextPagePos, pageSpeed * Time.deltaTime);
        transform.localScale = Vector3.Lerp(transform.localScale, nextPageSize, pageSpeed * Time.deltaTime);
        yield return null;
    }
    transform.position = nextPagePos;
    transform.localScale = nextPageSize;

    Managers.Game.ContentsScene.TimerRoutine = Managers.Game.ContentsScene.TimerStart(WaitTime);
    StartCoroutine(Managers.Game.ContentsScene.TimerRoutine);

    isStart = false;
}
```

LastPeople 자기장(컨텐츠)

- 자기장은 총 5단계의 Page가 존재하며 최종 Page에 도달할 때 까지 계속해서 작아 집니다.
- ContentScene 상속받은 클래스가 게임 시작이 PageStart를 호출합니다.
- NextDestination() 함수에서 다음 위치와 크기를 설정합니다. 위치는 현재 자기장 크기의 1/4값으로 랜덤 위치를 설정합니다. 크기는 nextPage 인덱스의 따른 pageSize 배열에 있는 크기를 따라갑니다.
- PageCoruotine()을 통해 목적지와 크기에 대해 Lerp하게 이동하거나 조절됩니다.
- 이동 및 조절이 끝나면 타이머를 작동시키며 타이머가 끝나면 위의 루틴을 반복합니다.

3주차 – 로딩씬, 게임씬, 콘텐츠 구현

콘텐츠 종료

```
public override void OnPlayerPropertiesUpdate(Player targetPlayer, ExitGames.Client.Photon.Hashtable changedProps)
{
    if (!changedProps.ContainsKey("Die") || !PhotonNetwork.IsMasterClient)
        return;

    if ((bool)changedProps["Die"])
        endEvent?.Invoke();
}
```

```
public abstract void GameEnd_M();
```

- 플레이어가 사망하거나 또는 승리 조건을 달성하면 해당 플레이어의 Properties를 수정하여 Photon의 콜백함수 OnPlayerPropertiesUpdate가 호출된다.
- 수정된 Properties가 조건에 맞는 Key를 포함하고 있고 마스터 클라이언트면 endEvent 델리케이트가 구독한 함수들을 실행하면 된다.
- OnPlayerPropertiesUpdate 함수는 가상 함수이며 해당 콘텐츠에 맞게 수정하면 된다.

3주차 – 로딩씬, 게임씬, 콘텐츠 구현

```
public class Enviroment : MonoBehaviour
{
    public Material skyBoxMaterial;
    public Light SunSource;
    public Vector3 LightRotation = new Vector3(50f, -180f, 0);
    public float LightIntensity = 1f;
    public Color LightColor = new Color(93f, 104f, 250f, 255f);
    public Color ShadowColor = new Color(107f, 122f, 160f, 255f);

    [Header("Enviroment Reflections")]
    public float reflectionIntensity = 1f;
    public int reflectionBounces = 1;

    [Header("Other Setting")]
    public bool isFog;
    public Color fogColor = new Color(128f, 128f, 128f, 255f);
    public FogMode fogMode = FogMode.ExponentialSquared;
    public float fogDensity = 0.01f;

    [Range(0f, 1f)]
    public float haloStrength = 1f;
    public float flareFadeSpeed = 3f;
    [Range(0f, 1f)]
    public float flareStrength = 3f;
}
```

```
private void Start()
{
    PV = GetComponent<PhotonViewEx>();

    if (PhotonNetwork.IsMasterClient)
    {
        if (isRandomWeather)
            targetWeatherNum = Random.Range(0, enviroments.Count);
        PV.RPC("ApplyWeather", RpcTarget.AllBufferedViaServer, targetWeatherNum);
    }

    [PunRPC]
    public void ApplyWeather(int index)
    {
        if (index >= enviroments.Count)
            return;

        targetWeatherNum = index;
        Enviroment enviroment = enviroments[index];

        RenderSettings.skybox = enviroment.skyBoxMaterial;
        enviroment.SunSource.transform.rotation = Quaternion.Euler(enviroment.LightRotation);
        enviroment.SunSource.intensity = enviroment.LightIntensity;
        enviroment.SunSource.color = enviroment.LightColor;
        RenderSettings.sun = enviroment.SunSource;
        RenderSettings.subtractiveShadowColor = enviroment.ShadowColor;

        RenderSettings.reflectionIntensity = enviroment.reflectionIntensity;
        RenderSettings.reflectionBounces = enviroment.reflectionBounces;

        RenderSettings.fog = enviroment.isFog;
        if (enviroment.isFog)
        {
            RenderSettings.fogColor = enviroment.fogColor;
            RenderSettings.fogMode = enviroment.fogMode;
            RenderSettings.fogDensity = enviroment.fogDensity;
        }

        RenderSettings.haloStrength = enviroment.haloStrength;
        RenderSettings.flareFadeSpeed = enviroment.flareFadeSpeed;
        RenderSettings.flareStrength = enviroment.flareStrength;
    }
}
```

환경광

- Enviroment 클래스 해당 콘텐츠 게임 씬의 환경광의 설정 값을 저장하는 클래스

EnviromentController 환경광 동기화

- 마스터 클라이언트가 Start함수 실행 시 enviroments 배열 중에서 랜덤으로 선택해서 ApplyWeather함수를 RPC해서 모든 클라이언트의 환경광을 동일하게 맞춰줍니다.

4주차 – 콘텐츠 구현 및 기타 수정

83 FPS (12.1 ms)



릿지 블록커

- 상단의 게이지가 다 닳을 때 까지 해당 캐릭터 색상이 맞는 버튼을 눌러 점수를 많이 획득하는 미니 게임
- 모든 플레이어가 탈락하면 게임 종료

4주차 – 콘텐츠 구현 및 기타 수정

119 FPS (8.4 ms)



슈팅 슈터

- 제한 시간안에 최대한 많은 플레이어를 섬멸
- 이동, 점프, 공격 단순한 조작
- 사망시 10초 뒤 리스폰

4주차 – 컨텐츠 구현 및 기타 수정

```
protected override void UpdateState()
{
    Input();

    switch (State)
    {
        case Define.State.Idle:
            UpdateIdle();
            break;
        case Define.State.Moving:
            UpdateMoving();
            break;
        case Define.State.NormalAttack:
            UpdateMoving();
            break;
        case Define.State.Jumping:
            UpdateJumping();
            break;
    }
}
```

슈팅 슈터 플레이어 FSM

- 플레이어와 AI 조작 로직은 FSM(유한 상태 머신)으로 구성되어 있습니다.
- 각 상태에 따라 유기적으로 연결되어 있습니다.

4주차 – 컨텐츠 구현 및 기타 수정

```
public override void OnPhotonSerializeView(PhotonStream stream, PhotonMessageInfo info)
{
    base.OnPhotonSerializeView(stream, info);

    if (stream.IsWriting)
    {
        stream.SendNext(transform.position);
        stream.SendNext(transform.rotation);
        stream.SendNext(sr.flipX);
        stream.SendNext(hpImage.fillAmount);
    }
    else
    {
        DestPos = (Vector3)stream.ReceiveNext();
        DestRot = (Quaternion)stream.ReceiveNext();
        sr.flipX = (bool)stream.ReceiveNext();
        hpImage.fillAmount = (float)stream.ReceiveNext();
    }
}
```

슈팅 슈터 플레이어 변수 동기화

- Photon의 콜백함수인 OnPhotonSerializeView 함수는 고정된 프레임 마다 각 클라이언트에게 값을 전달하여 동기화 합니다.
- 쓰기 상태는 오브젝트의 소유자가 실행하는 영역
- 읽기 상태 오브젝트의 소유자 제외 클라이언트가 실행하는 영역
- 고정된 프레임 마다 호출하기에 반응이 빠르다는 장점이 있습니다.

4주차 – 콘텐츠 구현 및 기타 수정

119 FPS (8.4 ms)



좀비 서바이벌

- 좀비를 피해 최대한 생존하는 미니게임
- 캐릭터의 좌클릭으로 좀비나 플레이어를 밀어낼 수 있습니다.
- 최후의 플레이어만 생존하거나 제한 시간안에 버티면 점수를 획득합니다.

4주차 – 컨텐츠 구현 및 기타 수정

```
protected override void UpdateState()
{
    Input();

    switch (State)
    {
        case Define.State.Idle:
            UpdateIdle();
            break;
        case Define.State.Moving:
            UpdateMoving();
            break;
        case Define.State.Run:
            UpdateRunning();
            break;
        case Define.State.NormalAttack:
            UpdateAttack();
            break;
        case Define.State.Jumping:
            UpdateJumping();
            break;
    }

    moveDir = GetCorrectionDirection(nextFrameMSpeed);
    ControlGravity();
}
```

```
protected override void UpdateState()
{
    switch (State)
    {
        case Define.State.Idle:
            UpdateIdle();
            break;
        case Define.State.Moving:
            UpdateMoving();
            break;
        case Define.State.NormalAttack:
            UpdateAttack();
            break;
    }
}
```

좀비 서바이벌 FSM

- 플레이어와 AI 조작 로직은 FSM(유한 상태 머신)으로 구성되어 있습니다.
- 각 상태에 따라 유기적으로 연결되어 있습니다.

4주차 – 컨텐츠 구현 및 기타 수정

```
[CreateAssetMenu(menuName = "AdditionPhysicsEffect/Effect")]
public class PhysicsEffect : ScriptableObject
{
    public Define.PhysicsType type;
    public Define.PhysicsDir dir;
    public float power;
    public float duraiton;
}
```

```
protected void OnAdditionPhysiced(PhotonViewEx targetPV)
{
    if (pEffect == null)
        return;

    for (int i = 0; i < pEffect.Length; i++)
    {
        targetPV.RPC("SetPhysics", targetPV.Owner, bc.PV.ViewID, (int)pEffect[i].type,
            (int)pEffect[i].dir, pEffect[i].power, pEffect[i].duraiton);
    }
}
```

```
[PunRPC]
public override void SetPhysics(int hitPlayerId, int type,int dir, float pow, float duration)
```

좀비 서바이벌 무기 효과

- 플레이어의 무기는 공격 시 해당 객체를 뒤로 날려버리는 특수 효과가 있습니다.
- Weapon클래스는 PhysicsEffect를 가질 수 있는 List 변수를 가지며 공격시 배열을 순회하면서 특수 효과를 적용 시킵니다. Ex) 밀치기
- BaseController에 선언되어 있는 SetPhyscis에서 매개변수로 온 값을 가지고 효과를 적용시킵니다.

개발에 사용된 에셋

- StylizedWater2
- Wooden_UI
- SimpleApocalypse
- ShootingShooter
- Fantastic UI Starter Pack
- Footsteps Sound Pack
- FREE Casual Game SFX Pack
- Game Music Pack - SUITE
- Photon
- Cinemachine
- RPG Inventory Icons
- Ultimate Sound FX Bundle
- LedgeBlocker
- etc...

구현에 어려웠던 점

- 플레이어를 이동, 3인칭 카메라 구현 많은 어려움이 있었습니다.
- URP(Universal Rendering PipeLine)을 써서 그래픽 옵션 설정 구현이 까다로웠고, 외부 라이브러리의 렌더링 기능이 추가되지 않아서 발생한 버그를 수정 하는게 많이 어려웠습니다. (바다에 들어갔을때 카메라가 바다에 들어간 부분만 추가 렌더링 하는 기능)
- 게임 컨텐츠 구조를 설계하는데 많이 어려웠습니다. (게임 대기 씬 ↔ 게임 씬)
- 다양한 미니게임을 만들기위한 2D, 3D 클래스를 생성하고 재사용하도록 코드를 작성하는게 어려웠습니다.

해결

- Youtube 영상을 참고하여 3인칭 플레이어, 카메라의 움직임을 구현하였습니다.
- Forward Renderer Data의 Add Renderer Feature를 통해 해당 Render기능을 추가하여 해결하였습니다.
- 공통적으로 사용하는 기능과 해당 씬에서만 사용되는 기능으로 나누어 구조를 설계하고 구현하였습니다.
- 많은 시행착오 끝에 2D, 3D 미니게임 콘텐츠에 사용되는 클래스를 구현과 코드간 재사용이 되도록 함수를 구현했습니다.

개발 후 느낀점 혹은 추후 개선 방향

- SaaS 로 만드는 멀티플레이어 게임의 한계를 느꼈다. 게임 판정을 클라이언트(마스터)가 관리하니까 코드 설계가 너무 어렵다. 예외적인 마스터가 방을 떠나거나 마스터가 바뀔때 마스터가 가지고 있는 데이터 동기화나 최적화를 위해 마스터만 실행하고 가지고있는 데이터를 다음 마스터가 가지게 하거나, 해당 상태에서 마스터가 나가면 될 예외처리 등등 너무 많은 상황을 처리해야해서 머리가 아프다. 그래서 소켓 프로그래밍을 해 직접 서버를 제작을 하는게 명확하게 프로그래밍이 가능할거라 생각한다.